

Breast Cancer

October 7, 2026

```
[13]: from mpl_toolkits.mplot3d import Axes3D
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt # plotting
import numpy as np # linear algebra
import os # accessing directory structure
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

import time
import random
import seaborn as sns
from sklearn.metrics import roc_curve, auc
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.model_selection import KFold, cross_val_score
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from urllib.request import urlopen

plt.style.use('ggplot')
pd.set_option('display.max_columns', 500)
```

```
[2]: bc = pd.read_csv("data (1).csv")
bc.head(5)
```

```
[2]:
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave	points_mean	\
0	0.11840	0.27760	0.3001		0.14710	
1	0.08474	0.07864	0.0869		0.07017	
2	0.10960	0.15990	0.1974		0.12790	
3	0.14250	0.28390	0.2414		0.10520	
4	0.10030	0.13280	0.1980		0.10430	

	symmetry_mean	fractal_dimension_mean	radius_se	texture_se	perimeter_se	\
0	0.2419	0.07871	1.0950	0.9053	8.589	
1	0.1812	0.05667	0.5435	0.7339	3.398	
2	0.2069	0.05999	0.7456	0.7869	4.585	
3	0.2597	0.09744	0.4956	1.1560	3.445	
4	0.1809	0.05883	0.7572	0.7813	5.438	

	area_se	smoothness_se	compactness_se	concavity_se	concave points_se	\
0	153.40	0.006399	0.04904	0.05373	0.01587	
1	74.08	0.005225	0.01308	0.01860	0.01340	
2	94.03	0.006150	0.04006	0.03832	0.02058	
3	27.23	0.009110	0.07458	0.05661	0.01867	
4	94.44	0.011490	0.02461	0.05688	0.01885	

	symmetry_se	fractal_dimension_se	radius_worst	texture_worst	\
0	0.03003	0.006193	25.38	17.33	
1	0.01389	0.003532	24.99	23.41	
2	0.02250	0.004571	23.57	25.53	
3	0.05963	0.009208	14.91	26.50	
4	0.01756	0.005115	22.54	16.67	

	perimeter_worst	area_worst	smoothness_worst	compactness_worst	\
0	184.60	2019.0	0.1622	0.6656	
1	158.80	1956.0	0.1238	0.1866	
2	152.50	1709.0	0.1444	0.4245	
3	98.87	567.7	0.2098	0.8663	
4	152.20	1575.0	0.1374	0.2050	

	concavity_worst	concave points_worst	symmetry_worst	\
0	0.7119	0.2654	0.4601	
1	0.2416	0.1860	0.2750	
2	0.4504	0.2430	0.3613	
3	0.6869	0.2575	0.6638	
4	0.4000	0.1625	0.2364	

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

```
[3]: # Distribution graphs (histogram/bar graph) of column data
def plotPerColumnDistribution(bc, nGraphShown, nGraphPerRow):
    nunique = bc.nunique()
```

```

    bc = bc[[col for col in bc if nunique[col] > 1 and nunique[col] < 50]] #
    ↪For displaying purposes, pick columns that have between 1 and 50 unique
    ↪values
    nRow, nCol = bc.shape
    columnNames = list(bc)
    nGraphRow = (nCol + nGraphPerRow - 1) // nGraphPerRow
    plt.figure(num = None, figsize = (6 * nGraphPerRow, 8 * nGraphRow), dpi =
    ↪80, facecolor = 'w', edgecolor = 'k')
    for i in range(min(nCol, nGraphShown)):
        plt.subplot(nGraphRow, nGraphPerRow, i + 1)
        columnbc = bc.iloc[:, i]
        if (not np.issubdtype(type(columnbc.iloc[0]), np.number)):
            valueCounts = columnbc.value_counts()
            valueCounts.plot.bar()
        else:
            columnbc.hist()
        plt.ylabel('counts')
        plt.xticks(rotation = 90)
        plt.title(f'{columnNames[i]} (column {i})')
    plt.tight_layout(pad = 1.0, w_pad = 1.0, h_pad = 1.0)
    plt.show()

```

```

[4]: # Correlation matrix
def plotCorrelationMatrix(bc, graphWidth):
    bc = bc[[col for col in bc if bc[col].nunique() > 1]] # keep columns where
    ↪there are more than 1 unique values
    if bc.shape[1] < 2:
        print(f'No correlation plots shown: The number of non-NaN or constant
        ↪columns ({bc.shape[1]} is less than 2')
        return
    corr = bc.corr()
    plt.figure(num=None, figsize=(graphWidth, graphWidth), dpi=80,
    ↪facecolor='w', edgecolor='k')
    corrMat = plt.matshow(corr, fignum = 1)
    plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
    plt.yticks(range(len(corr.columns)), corr.columns)
    plt.gca().xaxis.tick_bottom()
    plt.colorbar(corrMat)
    plt.title('Correlation Matrix', fontsize=15)
    plt.show()

```

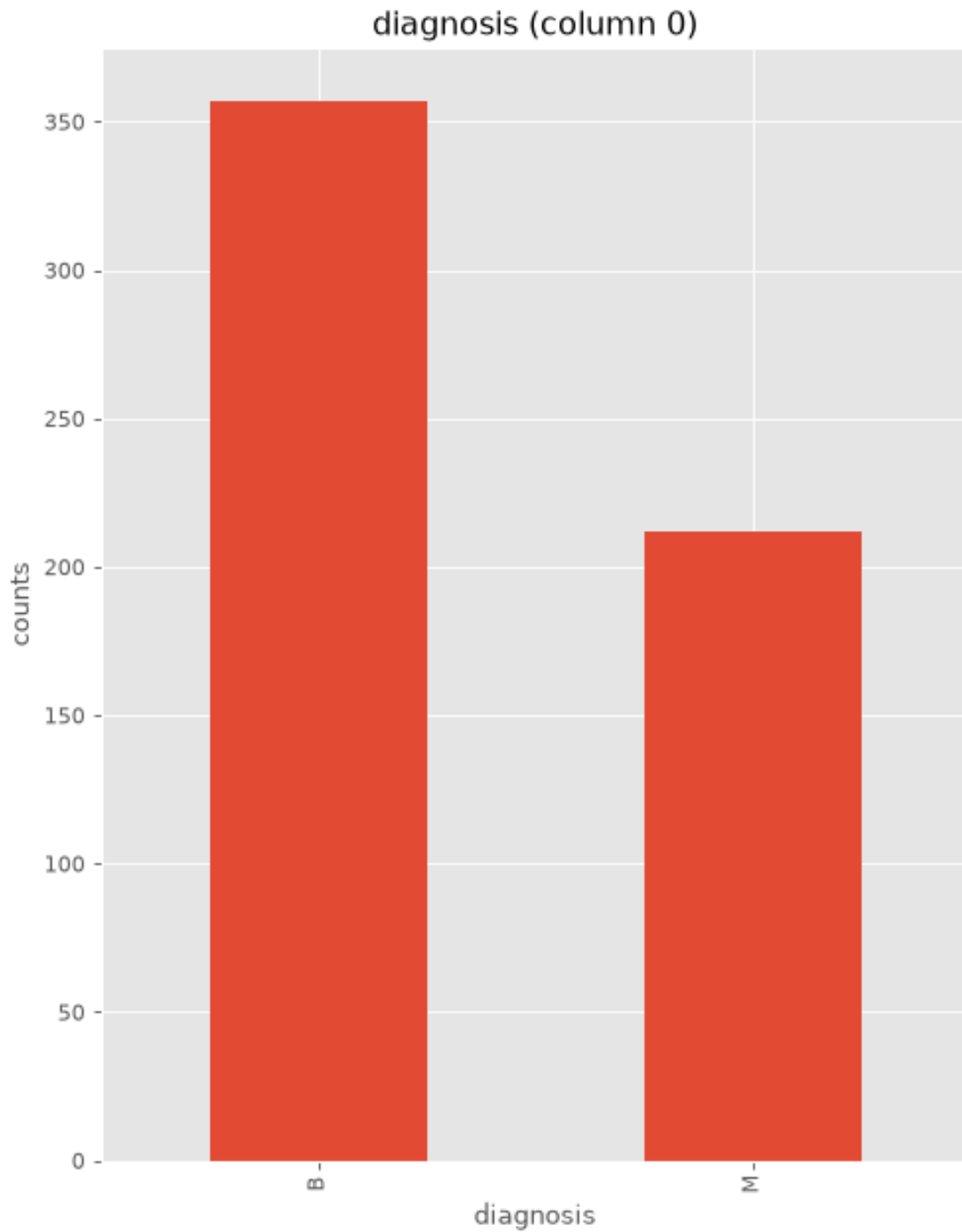
```

[5]: # Scatter and density plots
def plotScatterMatrix(bc, plotSize, textSize):
    bc = bc[[col for col in bc if bc[col].nunique() > 1]] # keep columns where
    ↪there are more than 1 unique values
    columnNames = list(bc)

```

```
if len(columnNames) > 10: # reduce the number of columns for matrix
    ↪inversion of kernel density plots
    columnNames = columnNames[:10]
    bc = bc[columnNames]
    ax = pd.plotting.scatter_matrix(bc, alpha=0.75, figsize=[plotSize,
    ↪plotSize], diagonal='kde')
    plt.suptitle('Scatter and Density Plot')
    plt.show()
```

```
[6]: plotPerColumnDistribution(bc, 10, 2)
```



```
[10]: # Setting 'id_number' as our index
bc=bc.set_index(['id'])
# Converted to binary to help later on with models and plots
bc['diagnosis'] = bc['diagnosis'].map({'M':1, 'B':0})
```

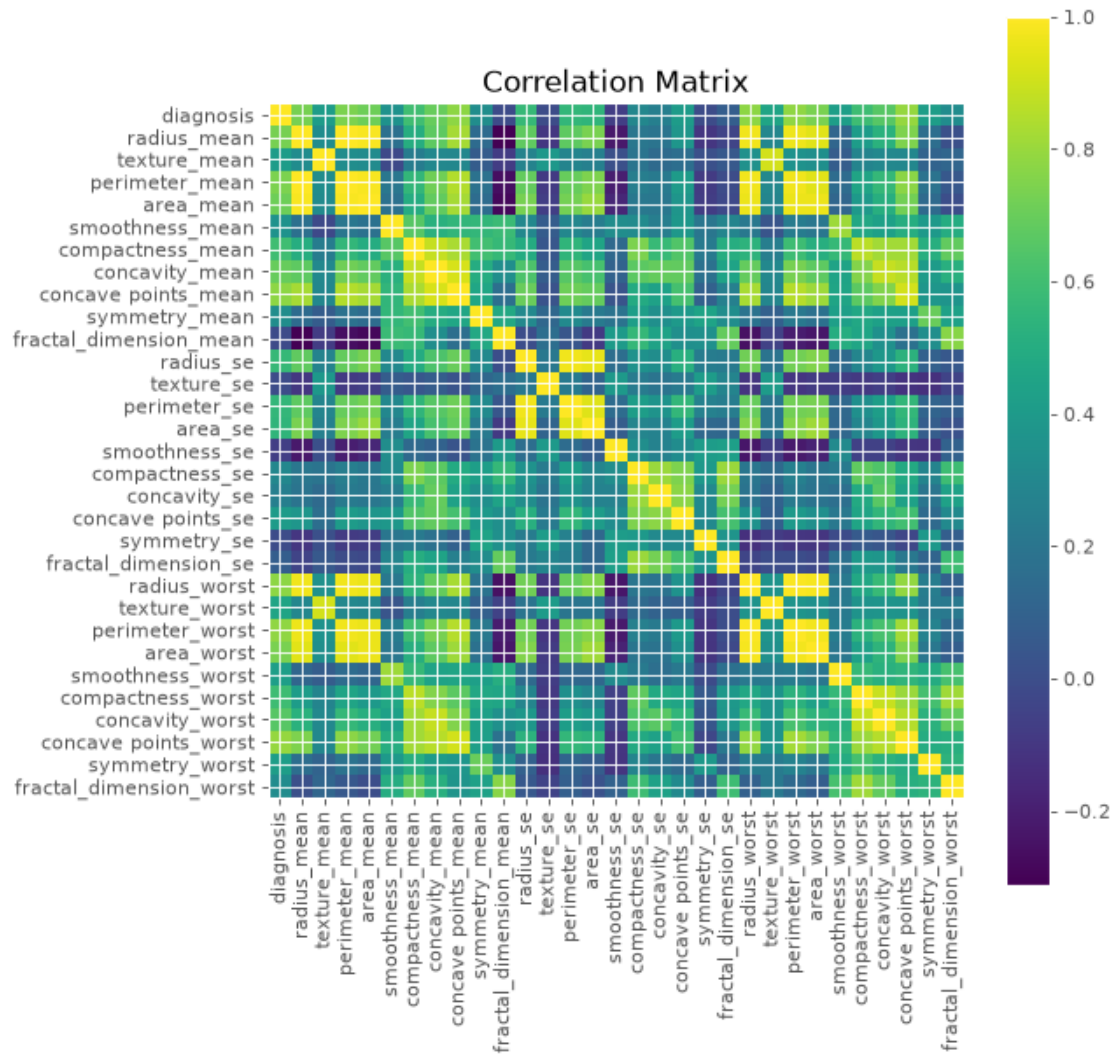
```
del bc['Unnamed: 32']
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[10], line 2
      1 # Setting 'id_number' as our index
----> 2 bc=bc.set_index(['id'])
      3 # Converted to binary to help later on with models and plots
      4 bc['diagnosis'] = bc['diagnosis'].map({'M':1, 'B':0})
      5 del bc['Unnamed: 32']

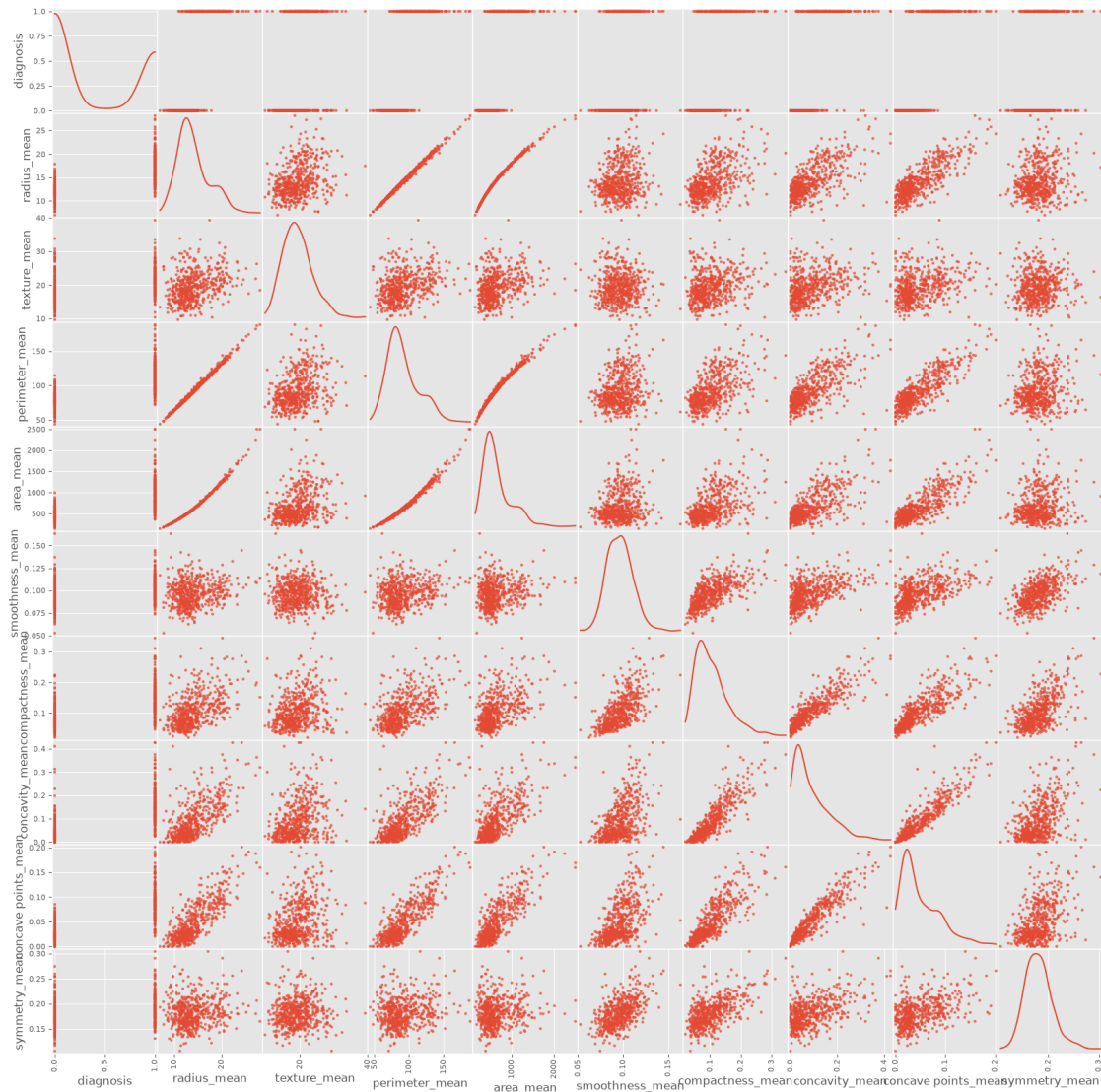
File /opt/conda/lib/python3.13/site-packages/pandas/core/frame.py:6989, in DataFrame.set_index(self, keys, drop, append, inplace, verify_integrity)
    6985         if not found:
    6986             missing.append(col)
    6987
    6988     if missing:
-> 6989         raise KeyError(f"None of {missing} are in the columns")
    6990
    6991     if inplace:
    6992         frame = self

KeyError: "None of ['id'] are in the columns"
```

```
[8]: plotCorrelationMatrix(bc, 8)
      plotScatterMatrix(bc, 20, 10)
```



Scatter and Density Plot



[9]: `bc.describe()`

```
[9]:
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	0.372583	14.127292	19.289649	91.969033	654.889104	
std	0.483918	3.524049	4.301036	24.298981	351.914129	
min	0.000000	6.981000	9.710000	43.790000	143.500000	
25%	0.000000	11.700000	16.170000	75.170000	420.300000	
50%	0.000000	13.370000	18.840000	86.240000	551.100000	

75%	1.000000	15.780000	21.800000	104.100000	782.700000
max	1.000000	28.110000	39.280000	188.500000	2501.000000

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
count	569.000000	569.000000	569.000000	569.000000	
mean	0.096360	0.104341	0.088799	0.048919	
std	0.014064	0.052813	0.079720	0.038803	
min	0.052630	0.019380	0.000000	0.000000	
25%	0.086370	0.064920	0.029560	0.020310	
50%	0.095870	0.092630	0.061540	0.033500	
75%	0.105300	0.130400	0.130700	0.074000	
max	0.163400	0.345400	0.426800	0.201200	

	symmetry_mean	fractal_dimension_mean	radius_se	texture_se	\
count	569.000000	569.000000	569.000000	569.000000	
mean	0.181162	0.062798	0.405172	1.216853	
std	0.027414	0.007060	0.277313	0.551648	
min	0.106000	0.049960	0.111500	0.360200	
25%	0.161900	0.057700	0.232400	0.833900	
50%	0.179200	0.061540	0.324200	1.108000	
75%	0.195700	0.066120	0.478900	1.474000	
max	0.304000	0.097440	2.873000	4.885000	

	perimeter_se	area_se	smoothness_se	compactness_se	concavity_se	\
count	569.000000	569.000000	569.000000	569.000000	569.000000	
mean	2.866059	40.337079	0.007041	0.025478	0.031894	
std	2.021855	45.491006	0.003003	0.017908	0.030186	
min	0.757000	6.802000	0.001713	0.002252	0.000000	
25%	1.606000	17.850000	0.005169	0.013080	0.015090	
50%	2.287000	24.530000	0.006380	0.020450	0.025890	
75%	3.357000	45.190000	0.008146	0.032450	0.042050	
max	21.980000	542.200000	0.031130	0.135400	0.396000	

	concave points_se	symmetry_se	fractal_dimension_se	radius_worst	\
count	569.000000	569.000000	569.000000	569.000000	
mean	0.011796	0.020542	0.003795	16.269190	
std	0.006170	0.008266	0.002646	4.833242	
min	0.000000	0.007882	0.000895	7.930000	
25%	0.007638	0.015160	0.002248	13.010000	
50%	0.010930	0.018730	0.003187	14.970000	
75%	0.014710	0.023480	0.004558	18.790000	
max	0.052790	0.078950	0.029840	36.040000	

	texture_worst	perimeter_worst	area_worst	smoothness_worst	\
count	569.000000	569.000000	569.000000	569.000000	
mean	25.677223	107.261213	880.583128	0.132369	
std	6.146258	33.602542	569.356993	0.022832	

min	12.020000	50.410000	185.200000	0.071170
25%	21.080000	84.110000	515.300000	0.116600
50%	25.410000	97.660000	686.500000	0.131300
75%	29.720000	125.400000	1084.000000	0.146000
max	49.540000	251.200000	4254.000000	0.222600

	compactness_worst	concavity_worst	concave points_worst	\
count	569.000000	569.000000	569.000000	
mean	0.254265	0.272188	0.114606	
std	0.157336	0.208624	0.065732	
min	0.027290	0.000000	0.000000	
25%	0.147200	0.114500	0.064930	
50%	0.211900	0.226700	0.099930	
75%	0.339100	0.382900	0.161400	
max	1.058000	1.252000	0.291000	

	symmetry_worst	fractal_dimension_worst
count	569.000000	569.000000
mean	0.290076	0.083946
std	0.061867	0.018061
min	0.156500	0.055040
25%	0.250400	0.071460
50%	0.282200	0.080040
75%	0.317900	0.092080
max	0.663800	0.207500

```
[14]: features = bc.iloc[:, bc.columns != 'diagnosis']
diagnosis = bc.iloc[:, bc.columns == 'diagnosis']
training_set, test_set, diagnosis_set, test_diagnosis_set = \
    ↪train_test_split(features, diagnosis,
                                test_size = \
                                ↪0.20, random_state = 1)
```

```
[16]: diagnosis_set = diagnosis_set.values.ravel()
test_diagnosis_set = test_diagnosis_set.values.ravel()
```

```
[17]: fit_rf = RandomForestClassifier(random_state=42)
```

```
[18]: np.random.seed(42)
start = time.time()

param_dist = {'max_depth': [2, 3, 4],
              'bootstrap': [True, False],
              'max_features': ['auto', 'sqrt', 'log2', None],
              'criterion': ['gini', 'entropy']}

cv_rf = GridSearchCV(fit_rf, cv = 10,
```

```

        param_grid=param_dist,
        n_jobs = 3)

cv_rf.fit(training_set, class_set)
print('Best Parameters using grid search: \n',
      cv_rf.best_params_)
end = time.time()
print('Time taken in grid search: {0: .2f}'.format(end - start))

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[18], line 13
      9 cv_rf = GridSearchCV(fit_rf, cv = 10,
     10                       param_grid=param_dist,
     11                       n_jobs = 3)
     12
----> 13 cv_rf.fit(training_set, class_set)
     14 print('Best Parameters using grid search: \n',
     15       cv_rf.best_params_)
     16 end = time.time()

NameError: name 'class_set' is not defined

```

```

[ ]: fit_rf.set_params(criterion = 'gini',
                      max_features = 'log2',
                      max_depth = 3)

```